

stm32wle5 te lorawan uygulamasında rx pencerei süresinde hem ackları hemde cevap mesajlarını kaçıyorum

1m 4s düşündü

Evet, burada iki farklı problemi birbirinden ayırmak gerekiyor. Senin mevcut `ImHandlerSend()` kullanım biçimini de düşününce, **ilk olarak uygulama akışını düzeltmemiz gerekiyor; sonra gerçekten RX timing problemi var mı onu ölçeriz.**

1. ACK ile uygulama cevabı aynı şey değil

Confirmed uplink gönderdiğinde:

```
STM32                Network Server
|                    |
|---- Confirmed UP ---->|
|                    |
|      ~1 saniye      |
|<---- RX1 Downlink ----|
|                    |
|      ~2 saniye      |
|<---- RX2 Downlink ----|   sadece RX1'de geçerli paket alınmadıysa
```

En önemli nokta şu:

RX1'de cihaz kendisine ait geçerli bir downlink aldıysa RX2 açılmaz.

Bu LoRaWAN Class A standardının doğrudan gereğidir.  LoRa Allianc... +1

Dolayısıyla şu senaryo **yanlış beklenti**:

```
RX1 -> ACK
RX2 -> uygulama cevabı
```

RX1'de ACK geldiyse RX2 açılmaz.

Bunun yerine network server mümkünse şunu göndermelidir:

```
RX1:
+-----+
| ACK = 1          |
| FPort = 10        |
| FRMPayload = RESPONSE |
+-----+
```

Yani **ACK biti + uygulama cevabı aynı downlink paketinde bulunabilir.**

2. ACK'yi OnRxData() içinde arama

STM32 LmHandler tarafında ACK kontrol edilmesi gereken yer:

C

```
static void OnTxData(LmHandlerTxParams_t *params)
{
    if (params->IsMcpsConfirm)
    {
        APP_LOG(TS_OFF, VLEVEL_M,
                "TX STATUS=%d ACK=%d FCnt=%lu\r\n",
                params->Status,
                params->AckReceived,
                params->UplinkCounter);

        if (params->MsgType == LORAMAC_HANDLER_CONFIRMED_MSG)
        {
            if (params->AckReceived)
            {
                APP_LOG(TS_OFF, VLEVEL_M, "ACK RECEIVED\r\n");
            }
            else
            {
                APP_LOG(TS_OFF, VLEVEL_M, "ACK NOT RECEIVED\r\n");
            }
        }
    }
}
```

LmHandlerTxParams_t içerisinde doğrudan:

C

```
uint8_t AckReceived;
```

alanı var.  GitHub +1

Uygulama payload'ı ise ayrı callback'ten gelir:

C

```
static void OnRxData(LmHandlerAppData_t *appData,
                    LmHandlerRxParams_t *params)
{
    APP_LOG(TS_OFF, VLEVEL_M,
            "DOWNLINK: slot=%d RSSI=%d SNR=%d port=%d len=%d\r\n",
            params->RxSlot,
            params->Rssi,
```

```

        params->Snr,
        appData->Port,
        appData->BufferSize);

    for (uint16_t i = 0; i < appData->BufferSize; i++)
    {
        APP_LOG(TS_OFF, VLEVEL_M, "%02X ", appData->Buffer[i]);
    }

    APP_LOG(TS_OFF, VLEVEL_M, "\r\n");
}

```

ST de `OnTxData()` callback'ini son TX durumunu, `OnRxData()` callback'ini ise alınan uygulama verisini işlemek için kullanıyor.  STMicroelectro...

3. Senin `for + LmHandlerSend()` kullanımın kesinlikle değişmeli

Önceki sorunundaki gibi:

```

C

for (int i = 0; i < 10; i++)
{
    LmHandlerSend(...);
}

```

yapamazsın.

Çünkü ilk:

```

C

LmHandlerSend()

```

çağrısından sonra LoRaMAC yaklaşık olarak şu durumda:

```

TX
↓
TX_DONE
↓
RX1 bekleniyor
↓
RX1
↓
gerekirse RX2
↓

```

MCPS Confirm

↓

OnTxData()

Bu işlem bitmeden ikinci `LmHandlerSend()` :

MAC BUSY

durumuna girer.

LoRaWAN standardı da önceki uplink'in RX işlemi tamamlanmadan yeni uplink gönderilmemesini şart koyuyor.  LoRa Allianc...

4. Doğru yapı: Queue + OnTxData

Senin projende özellikle circular buffer zaten olduğu için en uygun mimari bu.

Circular Buffer

|

v

SendNextPacket()

|

v

LmHandlerSend()

|

v

TX

|

+----- RX1

|

+----- RX2

|

v

OnTxData()

|

+- ACK varsa -> sent

|

+- ACK yoksa -> retry

|

v

UTIL_SEQ_SetTask(SendNextPacket)

Örneğin:

C

```
static bool LoraTxBusy = false;
```

Gönderim:

C

```
static void SendNextPacket(void)
{
    if (LoraTxBusy)
        return;

    LmHandlerAppData_t appData;

    appData.Port      = 10;
    appData.Buffer     = AppDataBuffer;
    appData.BufferSize = AppDataSize;

    LmHandlerErrorStatus_t status;

    status = LmHandlerSend(&appData,
                          LORAMAC_HANDLER_CONFIRMED_MSG);

    if (status == LORAMAC_HANDLER_SUCCESS)
    {
        LoraTxBusy = true;

        APP_LOG(TS_OFF, VLEVEL_M,
                "SEND REQUEST ACCEPTED\r\n");
    }
    else
    {
        APP_LOG(TS_OFF, VLEVEL_M,
                "SEND REQUEST FAILED\r\n");
    }
}
```

Sonra:

C

```
static void OnTxData(LmHandlerTxParams_t *params)
{
    if (!params->IsMcpsConfirm)
        return;

    APP_LOG(TS_OFF, VLEVEL_M,
            "TX DONE FCNT=%lu ACK=%d STATUS=%d\r\n",
            params->UplinkCounter,
            params->AckReceived,
            params->Status);

    if (params->MsgType == LORAMAC_HANDLER_CONFIRMED_MSG)
    {

```

```

if (params->AckReceived)
{
    /*
     * Circular buffer içerisindeki
     * mevcut paketi SENT yap.
     */
    MarkCurrentPacketAsSent();
}
else
{
    /*
     * UNSENT olarak bırak.
     * Retry mekanizması sonra tekrar gönderebilir.
     */
}
}

LoraTxBusy = false;

/*
 * Doğrudan burada LmHandlerSend() çağırma.
 * Sequencer üzerinden foreground task çalıştır.
 */
UTIL_SEQ_SetTask(
    (1 << CFG_SEQ_Task_LoRaSendNextEvent),
    CFG_SEQ_Prio_0);
}

```

Bu senin cihazın için `for` döngüsünden çok daha doğru yapı.

5. Bundan sonra gerçekten RX kaçırıyor muyuz test edelim

Yukarıdaki mimariyi yaptıktan sonra hâlâ:

ACK=0

görüyorsan o zaman **gerçek RX problemi** araştırırız.

İlk yapacağım test:

C

```
LmHandlerSetSystemMaxRxError(100);
```

Normal ST/LoRaMac örneklerinde bu değer genellikle daha düşük tutuluyor; API doğrudan RX timing toleransını genişletmek için mevcut.

Debug amacıyla önce:

C

```
LmHandlerSetSystemMaxRxError(100);
```

hatta kısa süreliğine:

C

```
LmHandlerSetSystemMaxRxError(200);
```

dene.

Eğer:

```
20 ms -> ACK kaçıyor  
100 ms -> ACK geliyor
```

gibi bir sonuç çıkarsa sorun büyük ihtimalle:

```
RTC / LSE  
timer_if.c  
STOP2 wake-up timing  
radio wake-up timing
```

tarafındadır.

Kalıcı olarak gereksiz büyük yapmayız çünkü RX penceresi büyür ve batarya tüketimi artar. LoRaMac-node tarafında da bu fonksiyonun özellikle clock/timing hatalarını teşhis etmek amacıyla büyütülebileceği belirtiliyor. [GitHub](#)

6. TX ile RX2 tamamlanana kadar bunları özellikle yapma

Senin projedeki bileşenleri düşünersek özellikle şunlara dikkat:

```
LmHandlerSend()  
|  
+---- HAL_Delay(...)  
|  
+---- MFRC522 uzun polling  
|  
+---- Flash erase  
|
```

```
+---- uzun SPI işlemi
|
+---- while(...)
|
+---- __disable_irq()
|
+---- ikinci LmHandlerSend()
```

bunların hiçbirini RX kritik zamanında istemiyorum.

Özellikle **Flash page erase** veya uzun interrupt-disabled kritik bölge varsa RX timing'i mahvedebilir.

Normal yapı:

C

```
LmHandlerSend();

return;
```

olmalı.

Ardından kontrol tekrar:

C

```
MX_LoRaWAN_Process();
```

ve `UTIL_SEQ_Run()` / `LmHandlerProcess()` tarafına dönmeli. `LmHandlerProcess()` zaten ana döngüde sürekli işlenmesi gereken fonksiyon olarak tanımlanmış.  [GitHub](#)

7. Server cevabı konusu ayrıca çok önemli

Örneğin sen:

```
STM32
|
| uplink
v
Gateway
|
Network Server
|
MQTT
|
Senin Server
```

```
|  
MQTT downlink  
|  
Network Server
```

şeklinde cevap üretiyorsan, bu round-trip süresi **mevcut uplink'in RX1'ine yetişmeyebilir**.

Network ACK'i çok hızlı oluşturabilir:

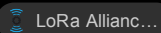
```
Confirmed Uplink  
|  
+-- Network Server  
|  
+-- ACK -> RX1
```

ama uygulama cevabı:

```
uplink  
↓  
MQTT  
↓  
backend  
↓  
işlem  
↓  
MQTT publish  
↓  
network server
```

yolundan geldiğinde RX1 geçmiş olabilir.

Daha da önemlisi ACK zaten RX1'de alındıysa STM32 **RX2'yi açmayacaktır**.



Bu durumda doğru akış:

```
UPLINK #10  
↓  
RX1 = ACK  
  
server response queue'da kalır  
  
UPLINK #11  
↓  
RX1 = RESPONSE
```

veya network server cevabı zamanında biliyorsa:

```
UPLINK #10
↓
RX1 =
ACK=1
+
RESPONSE PAYLOAD
```

olmalı.

Ben senin durumda şu sırayla ilerlerdim

1. `for` içindeki bütün `LmHandlerSend()` çağrılarını kaldır.
2. Tek seferde **yalnızca bir uplink** gönder.
3. Sonraki paketi yalnızca `OnTxData()` sonrasında `UTIL_SEQ_SetTask()` ile gönder.
4. ACK kontrolünü kesinlikle `params->AckReceived` üzerinden yap.
5. `OnRxData()` içine `RxSlot/RSSI/SNR/FPort/Size` loglarını ekle.
6. Test amacıyla `LmHandlerSetSystemMaxRxError(100)` yap.
7. Network-server/gateway logunda downlink'in gerçekten hangi **RX slot, frekans ve DR** ile transmit edildiğini kontrol et.
8. Eğer gateway `TX ACK/downlink` gösteriyor ama STM32 `AckReceived=0` diyorsa artık doğrudan **STM32 RX timing / RF tarafına** odaklanınız.

Bence şu anda **en kuvvetli iki aday**, art arda `LmHandlerSend()` nedeniyle MAC state'in bozulması ve ACK ile uygulama response'unun RX1/RX2'de iki ayrı mesaj olarak beklenmesi. Bu ikisini düzelttikten sonra hâlâ `AckReceived=0` kalıyorsa `timer_if.c + RTC + SUBGHZ IRQ + RX window` seviyesine inmek gerekir.

Yapıştırılan markdown....
Dosya

2m 1s düşündü

Kodu inceledim. Burada RX1/RX2 kaçırılmasını açıklayan **somut bir concurrency/sequencer problemi var**. Hatta kodun içindeki yorumlarda join sırasında aynı probleme daha önce denk geldiğin zaten belirtilmiş: RFID okumasının yaklaşık 5–6 saniye bloklayıp `LmHandlerProcess()` çalışmasını geciktirebildiğini yazmışsın.

Yapıştırılan markdown

Asıl problem: `ReadRFIDCard()` **LoRa işlemi sürerken başlayabiliyor**

`ReadRFIDCard()` başında şu kontroller var:

C

```
if (rfid_data_pending_on_lora)
{
    return;
}

if (join_in_progress)
{
    return;
}
```

Ama şunlar kontrol edilmiyor:

C

```
status_data_pending_on_lora
buffered_rfid_data_wait_for_ack
```

Bu kritik.

Örneğin status mesajında önce:

C

```
status_data_pending_on_lora = true;

LmHandlerSend(
    &AppData,
    LORAMAC_HANDLER_CONFIRMED_MSG,
    false);
```

yapıyorsun.

Yapıştınlan markdown

Ardından kullanıcı butona basarsa `ReadRFIDCard()` çalışabiliyor.

Ve `ReadRFIDCard()` içinde kabaca:

C

```
HAL_Delay(5);

for(int i=0; i<5; i++)
{
    ...
    HAL_Delay(100);
}
```

```

while (!stop_read_rfid)
{
    waitcardDetectUntilTimeout(...);

    ...

    Buzzer_Alert_Process(1500);

    ...

    waitcardRemovalUntilTimeout(...);
}

HAL_Delay(5);

```

var.

Yani sequencer task'ı saniyeler boyunca geri dönmüyor.

ST'nin kendi dokümantasyonu burada çok net: `UTIL_SEQ` bir RTOS değildir ve **task run-to-completion çalışır**. Çalışmakta olan task bitmeden başka task'a geçilmez; task priority bile ancak çalışan fonksiyon bittikten sonra değerlendirilir.  ST...

Sen de `LmHandlerProcess` 'i sequencer task'ı olarak kaydetmişsin:

C

```

UTIL_SEQ_RegTask(
    (1 << CFG_SEQ_Task_LmHandlerProcess),
    UTIL_SEQ_RFU,
    LmHandlerProcess);

```

Yapıştıran markdown

Dolayısıyla şöyle bir olay oluşabiliyor:

```

Status LmHandlerSend()
|
v
TX başlıyor
|
v
SUBGHZ TX_DONE IRQ
|
v
OnMacProcessNotify()
|
v
LmHandlerProcess task READY

```

```
|
X
|
ReadRFIDCard() şu anda çalışıyor
while() /HAL_Delay() /MFRC522...
|
5 sn
|
v
LmHandlerProcess ancak şimdi çalışabiliyor
```

Ve buradaki en tehlikeli ayrıntı şu:

`TX_DONE` sonrasında RX1/RX2 timer'larının kurulması MAC processing içinde gerçekleşiyor. LoRaMac tarafında TX_DONE radio event'i işlendikten sonra RX1 ve RX2 timer'ları kuruluyor.  [GitHub](#) +1

Yani mesele yalnızca:

"RX callback geç işlendi"

değil.

Daha kötü senaryoda:

RX1/RX2'nin hazırlanması gereken MAC işlemi zamanında çalışmıyor.

Bu yüzden ACK de, uygulama downlink'i de kaçabilir.

Önce şu düzeltmeyi yap

En az değişiklikle `ReadRFIDCard()` başlangıcını şöyle yap:

```
C

static void ReadRFIDCard(void)
{
    if (join_in_progress ||
        rfid_data_pending_on_lora ||
        buffered_rfid_data_wait_for_ack ||
        status_data_pending_on_lora)
    {
        APP_LOG(TS_OFF, VLEVEL_M,
            "##### LoRa TX/RX devam ediyor, RFID okumasi ertelendi\r\n");

        return;
    }
}
```

```
/*  
 * RFID kodunun geri kalani  
 */
```

Bu deęişiklik **özellikle önemli**.

Zaten dięer yönde koruma koymuşsun. Örneęin status gönderirken RFID veya buffered ACK bekleniyorsa status'u erteliyorsun.

Yapıştırılan markdown

Buffer gönderiminde de başka bir RFID/status işlemi varsa çıkıyorsun.

Yapıştırılan markdown

`SendRFID_Data()` aşamasında da status/buffer pending ise RFID verisini LoRa'ya göndermeyip buffer'a alıyorsun.

Yapıştırılan markdown

Ama problem şu:

```
ReadRFIDCard()
```

bu korumalardan **önce**, yani fiziksel RFID okuması sırasında sistemi blokluyor.

Ben bir adım daha ileri giderdim: tek

```
lora_transaction_active
```

Şu anda dört-beş farklı bool ile LoRa state yönetiyorsun:

C

```
rfid_data_pending_on_lora  
buffered_rfid_data_wait_for_ack  
status_data_pending_on_lora  
join_in_progress
```

Bu bir süre sonra kaçınılmaz olarak açık bırakıyor.

Bunun yerine uygulama seviyesinde:

C

```
static volatile bool lora_transaction_active = false;  
static volatile bool rfid_read_deferred = false;
```

ekle.

Her `LmHandlerSend()` başarılı olduğunda:

C

```
LmHandlerErrorStatus_t status =  
    LmHandlerSend(&AppData,  
                  LORAMAC_HANDLER_CONFIRMED_MSG,  
                  false);  
  
if (status == LORAMAC_HANDLER_SUCCESS)  
{  
    lora_transaction_active = true;  
}
```

Aynısını:

C

```
OnStatusMessageHandler()  
SendRFID_Data()  
SendBufferedRfidLogHandler()  
SendTxData()
```

içindeki bütün gerçek uplink'lere uygula.

Sonra RFID başlangıcını böyle yap

C

```
static void ReadRFIDCard(void)  
{  
    if (join_in_progress || lora_transaction_active)  
    {  
        APP_LOG(TS_OFF, VLEVEL_M,  
                "##### LoRa işlemi devam ediyor, RFID ertelendi\r\n");  
  
        rfid_read_deferred = true;  
        return;  
    }  
  
    rfid_read_deferred = false;  
  
    /*  
    * RFID okuma...  
    */  
}
```

Bu çok daha temiz.

OnTxData () içinde transaction'ı bitir

Senin mevcut OnTxData () filtren doğru yönde:

C

```
if ((params != NULL) &&
    (params->IsMcpsConfirm != 0) &&
    (params->MsgType ==
     LORAMAC_HANDLER_CONFIRMED_MSG))
```

Ama transaction state'i daha yukarıda çöz:

C

```
static void OnTxData(LmHandlerTxParams_t *params)
{
    if ((params == NULL) ||
        (params->IsMcpsConfirm == 0))
    {
        return;
    }

    /*
     * Artık TX + RX1/RX2 transaction tamamlandı.
     */
    lora_transaction_active = false;

    APP_LOG(TS_OFF, VLEVEL_M,
            "MCPS DONE: FCnt=%lu ACK=%d status=%d\r\n",
            params->UplinkCounter,
            params->AckReceived,
            params->Status);

    /*
     * mevcut ACK işleme kodun...
     */

    if (rfid_read_deferred)
    {
        rfid_read_deferred = false;

        UTIL_SEQ_SetTask(
            (1 << CFG_SEQ_Task_ReadRFIDEvent),
            CFG_SEQ_Prio_0);
    }
}
```

Burada UTIL_SEQ_SetTask() kullanmak güvenli. ReadRFIDCard() hemen callback'in içinde başlamaz. LmHandlerProcess() tamamlanıp sequencer'a döndükten sonra çalışır.

Join de aynı transaction mantığına dahil edilmeli

TryJoin() :

C

```
join_in_progress = true;
```

zaten kullanıyor.

Dolayısıyla RFID kontrolü:

C

```
if (join_in_progress || lora_transaction_active)
```

olmalı.

Join bittiğinde:

C

```
static void OnJoinRequest(LmHandlerJoinParams_t *joinParams)
{
    join_in_progress = false;

    ...

    if (rfid_read_deferred)
    {
        rfid_read_deferred = false;

        UTIL_SEQ_SetTask(
            (1 << CFG_SEQ_Task_ReadRFIDEvent),
            CFG_SEQ_Prio_0);
    }
}
```

yapabilirsin.

Bir başka önemli nokta: RFID task'ı tasarım olarak yine tehlikeli

Yukarıdaki değişiklik mevcut sistemi büyük ölçüde düzeltecek. Ama daha doğru çözüm:

C

```
while (!stop_read_rfid)
```

kullanmamak.

ST'nin sequencer mimarisinde uzun çalışan task istemiyoruz. ST açıkça task'ların run-to-completion olduğunu söylüyor.  STMicroelectro...

Şu an:

```
ReadRFIDCard()  
├─ power on  
├─ HAL_Delay  
├─ LED 500 ms  
├─ while  
│   ├─ MFRC poll  
│   ├─ MFRC poll  
│   ├─ MFRC poll  
│   └─ ...  
└─ buzzer 1500 ms  
    └─ return
```

yerine ideal yapı:

```
ReadRFIDStart()  
    ↓  
RFID_POLL timer 50 ms  
    ↓  
ReadRFIDStep()  
    ↓  
kart yok → return  
    ↓  
50 ms sonra tekrar task  
    ↓  
kart bulundu  
    ↓  
RFID finish
```

olmalı.

Her task örneğin 1–5 ms çalışıp sequencer'a dönmeli.

O zaman:

```
RFID Step  
    ↓  
return  
    ↓  
LmHandlerProcess  
    ↓
```

```
return  
↓  
RFID Step
```

gibi LoRa ve RFID doğal biçimde beraber yaşayabilir.

`OnMacProcessNotify()` tarafın doğru

Şu bölüm:

C

```
static void OnMacProcessNotify(void)  
{  
    UTIL_SEQ_SetTask(  
        (1 << CFG_SEQ_Task_LmHandlerProcess),  
        CFG_SEQ_Prio_0);  
}
```

doğru yaklaşım.

Problem burası değil.

Problem şu:

```
SetTask(LmHandlerProcess)
```

yapılıyor ama sequencer o task'a geçemiyor.

Çünkü önündeki `ReadRFIDCard()` bitmemiş oluyor.

ST dokümanındaki özellikle şu cümle senin problemine birebir uyuyor: task priority yalnızca çalışan fonksiyon **bittikten sonra** değerlendirilir.  STMicroelectro...

Dolayısıyla:

C

```
CFG_SEQ_Prio_0
```

vermek problemi çözmez.

`RfidPreventStopMode()` sorun değil

Şu:

C

```
UTIL_LPM_SetStopMode(  
    RFID_LPM_USER_MASK,  
    UTIL_LPM_DISABLE);
```

RX kaçırmanın temel sebebi değil.

Bu sadece RFID sırasında MCU'nun STOP'a girmesini engelliyor.

Hatta LoRa açısından CPU'nun uyanık kalması sorun yaratmaz.

Dolayısıyla şimdilik:

C

```
RfidPreventStopMode();  
...  
RfidAllowStopMode();
```

yapısını değiştirmene gerek yok.

Bir problem daha: ACK ve TIME_SYNC aynı transaction

Status mesajında:

C

```
LmHandlerSend(  
    &AppData,  
    LORAMAC_HANDLER_CONFIRMED_MSG,  
    false);
```

kullanıyorsun.

Sonra `OnRxData()` içinde TIME_SYNC bekliyorsun.

Burada iki durum var:

```
STATUS confirmed uplink  
    ↓  
Network Server  
    ↓  
RX1
```

RX1 downlink'i:

```
ACK = 1
FRMPayload = TIME_SYNC
```

şeklinde gelirse:

```
OnTxData() -> AckReceived = 1
OnRxData() -> TIME_SYNC
```

ikisini de alırsın.

Ama:

```
RX1 -> yalnızca ACK
RX2 -> TIME_SYNC
```

beklentisi doğru değil.

RX1'de geçerli downlink alındığında Class A cihaz RX2'yi normalde açmaz.

ST de RX1 ve RX2'yi ayrı MAC reception-window timer'ları olarak tanımlıyor.



Eğer TIME_SYNC cevabını MQTT/backend üzerinden sonradan oluşturuyorsan:

```
uplink
↓
network server
↓
MQTT
↓
server
↓
processing
↓
MQTT downlink
```

RX1'e yetişmemesi ayrıca mümkün.

Bu durumda TIME_SYNC bir sonraki uplink'in RX penceresinde gelir.

`OnRxData()` **logunu da değiştir**

Şu an:

C

```
APP_LOG(...,  
    "D/L FRAME: port %d, %d byte(s), RSSI %d, SNR %d\r\n",  
    ...);
```

var.

Buna mutlaka `RxSlot` ekle:

C

```
APP_LOG(TS_OFF, VLEVEL_M,  
    "##### D/L: slot=%d port=%d len=%d RSSI=%d SNR=%d time=%lu\r\n",  
    (int)params->RxSlot,  
    appData->Port,  
    appData->BufferSize,  
    params->Rssi,  
    params->Snr,  
    (unsigned long)UTIL_TIMER_GetCurrentTime());
```

Böylece:

```
slot = RX1
```

mi,

```
slot = RX2
```

mi hemen görürüz.

Ayrıca timing logu koy

`ImHandlerSend()` başarılı olduğunda:

C

```
APP_LOG(TS_OFF, VLEVEL_M,  
    "TX ACCEPTED @ %lu\r\n",  
    (unsigned long)UTIL_TIMER_GetCurrentTime());
```

`ReadRFIDCard()` başında:

C

```
APP_LOG(TS_OFF, VLEVEL_M,
        "RFID START @ %lu\r\n",
        (unsigned long)UTIL_TIMER_GetCurrentTime());
```

sonunda:

C

```
APP_LOG(TS_OFF, VLEVEL_M,
        "RFID END @ %lu\r\n",
        (unsigned long)UTIL_TIMER_GetCurrentTime());
```

OnTxData() :

C

```
APP_LOG(TS_OFF, VLEVEL_M,
        "TX CONFIRM @ %lu ACK=%d\r\n",
        (unsigned long)UTIL_TIMER_GetCurrentTime(),
        params->AckReceived);
```

Bir problem olduğunda muhtemelen şöyle bir log göreceğiz:

```
TX ACCEPTED @ 100000
RFID START @ 100050

... 5 saniye ...

RFID END @ 105400

TX CONFIRM @ 105410 ACK=0
```

Bu çıkarsa teşhis kesinleşmiş olur.

Sonuç

Şu anda **RX window süresini büyötmeye veya** `LmHandlerSetSystemMaxRxError()` **ile oynamaya başlamazdım.**

Önce kodundaki bu problemi çözerim:

```
LoRa transaction
TX → RX1 → RX2 → OnTxData
      ↑
      |
BU SÜREDE RFID YASAK
```

Minimum düzeltme olarak hemen şunu ekle:

C

```
if (rfid_data_pending_on_lora ||
    buffered_rfid_data_wait_for_ack ||
    status_data_pending_on_lora ||
    join_in_progress)
{
    return;
}
```

`ReadRFIDCard()` başına.

Fakat kalıcı çözüm olarak ben **tek** `lora_transaction_active` **flag + deferred RFID task** yapısına geçdim. Ondan sonraki aşamada da `ReadRFIDCard()` içindeki bloklayıcı `while` + `HAL_Delay` yapısını küçük sequencer task'larından oluşan state machine'e dönüştürdüm.

Bu kodu gördükten sonra RX kaçırmamanın birinci şüphelisi artık RTC/LSE değil; **bloklayıcı RFID sequencer task'ı**.

Kaynaklar

Bu kişiliği sevdin mi?

ChatGPT hata yapabilir. Önemli bilgileri kontrol et.